CampusEats — Services, Contracts & Schema Design Benchmark

Assumption note: The Assignment 1 brief and actual team-member names were not included with this request. This benchmark therefore uses a standard campus food-ordering scope and placeholders Member A–D. Replace those names/capabilities with the exact Assignment 1 wording before submission.

1. Capabilities

• Register and authenticate campus users

• Manage food items and their availability

• Browse and search the food catalogue

• Create and manage food orders

• Calculate order totals and validate item availability

• Process and record payments

• Track order status and order history

• Provide order and payment information to the customer

2. Service Boundaries

Service	Owner	Data owned
Identity Service	Member A	users, auth_sessions
Catalogue Service	Member B	food_items
Order Service	Member C	orders, order_items
Payment Service	Member D	payments

Cross-service calls: Order → Identity: checkUser; Order → Catalogue: checkItem and getItemPrice; Order → Payment: authorizePayment and getPaymentStatus. No service shares ownership of another service's data.

3. Service Contracts

Identity Service

Operation	Input	Output	Errors
checkUser	user reference	User valid / invalid	USER_NOT_FOUND; USER_INACTIVE
authenticate	email, password	Authenticated user context	INVALID_CREDENTIALS; USER_INACTIVE
getUserProfile	user reference	Profile summary	USER_NOT_FOUND

Catalogue Service

Operation	Input	Output	Errors
checkItem	item reference, quantity	Availability result	ITEM_NOT_FOUND; ITEM_UNAVAILABLE; INVALID_QU
getItemPrice	item reference	Current price	ITEM_NOT_FOUND; ITEM_UNAVAILABLE
searchItems	query, optional category	Matching item summaries	INVALID_QUERY

Order Service

Operation	Input	Output	Errors
placeOrder	user reference, items, payment details	Order result	USER_INVALID; ITEM_UNAVAILABLE; PAYMENT_FAIL
getOrder	user reference, order reference	Order summary	ORDER_NOT_FOUND; ACCESS_DENIED
cancelOrder	user reference, order reference	Cancellation result	ORDER_NOT_FOUND; CANNOT_CANCEL; ACCESS_D

Payment Service

Operation	Input	Output	Errors
authorizePayment	order reference, amount, payment details	Payment authorization result	PAYMENT_DECLINED; INVALID_PAYMENT; PROVIDER
getPaymentStatus	payment reference	Payment status	PAYMENT_NOT_FOUND
refundPayment	payment reference, amount	Refund result	PAYMENT_NOT_FOUND; REFUND_REJECTED

4. Full Specification — placeOrder

Purpose: Create a food order for an authenticated campus user, verify each requested item, calculate the authoritative total, obtain payment authorization, and return the resulting order.

Inputs

• userRef — caller's authenticated user reference.

• items — one or more requested items, each containing itemRef and quantity.

• paymentDetails — payment method information required by the Payment Service.

Output

• orderRef — newly created order reference.

• status — initial order status.

• items — accepted items with names, quantities and prices.

• totalAmount — authoritative order total.

• paymentStatus — authorization result.

Error cases

• USER_INVALID — the caller is not a valid/active campus user.

• ITEM_NOT_FOUND — one requested item does not exist.

• ITEM_UNAVAILABLE — an item cannot currently be ordered.

• INVALID_QUANTITY — requested quantity is invalid.

• PAYMENT_FAILED — payment authorization was declined or failed.

• ORDER_REJECTED — the order could not be safely completed.

Internal details hidden from callers

• How orders and order items are persisted.

• How item prices and availability are stored or cached.

• How service-to-service authentication is implemented.

• How payment providers, transaction references, retries, timeouts and idempotency are handled.

• Database table names, primary-key formats, SQL queries, indexes and transaction mechanics.

Internal flow: authenticate/validate user → validate each item with Catalogue → obtain authoritative prices → calculate total → create pending order → request payment authorization → mark order confirmed on success or rejected on failure → return the contract response. The caller sees only the contract, not these implementation steps.

6. Service Validation

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled	Fix if needed
Identity	Yes	Yes	Yes	Yes	Yes	None
Catalogue	Yes	Yes	Yes	Yes	Yes	None
Order	Yes	Yes	Yes	Yes	Mostly	Use stable contract calls; never access Catalogue/Payment sto
Payment	Yes	Yes	Yes	Yes	Yes	None

Definitions: Reachable = other components can invoke it through a clear endpoint; self-contained = owns the data required for its responsibility; has a contract = operations expose behavior without implementation details; independent = can be changed/deployed without changing another service's internals; loosely coupled = dependencies are through stable contracts rather than shared storage.

Submission note

services.drawio and schema.drawio are editable diagrams. Export them from diagrams.net as PNG/PDF if the supplied PNGs are not generated automatically. Replace Member A–D and any assumptions with the actual Assignment 1 team/member information before committing.